Báo cáo tiến độ tuần từ ngày 29/3 đến ngày 4/4

💡 Viết các file test linh kiện cho gateway và SensorNode

Test của Gateway

File test_http.cpp:

```
/**
 * TEST: HTTP POST - Gửi JSON giả lên Cloud Server
 *
 * Flash: pio run -e test_http -t upload && pio device moni
tor
 *
 * Cần: Server backend đang chạy ở API_URL
 * PASS khi: Server trả về`200 OK
 */

#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include "config.h"

static uint16_t sendCount = 0;

void setup()
{
    Serial.begin(115200);
    delay(1000);

    Serial.println();
    Serial.println("========================================
=");
```

```
    Serial.println("  TEST: HTTP POST → Cloud Server");
    Serial.printf("  API: %s\n", API_URL);
    Serial.printf("  Gateway ID: %s\n", GATEWAY_ID);
    Serial.println("========================================
=");
    Serial.println();

    // Kết nối WiFi trước
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("[WiFi] Đang kết nối");

    unsigned long start = millis();
    while (WiFi.status() != WL_CONNECTED)
    {
        if (millis() - start > WIFI_CONNECT_TIMEOUT_MS)
        {
            Serial.println("\n[WiFi] ❌ Timeout! Không kết
nối được.");
            while (true)
                delay(1000);
        }
        Serial.print(".");
        delay(500);
    }
    Serial.printf("\n[WiFi] ✅ IP: %s\n\n", WiFi.localIP().
toString().c_str());

    Serial.println("[HTTP] Gửi JSON test mỗi 10 giây...");
    Serial.println("————————————————————————————————————
—");
}

void loop()
{
    if (WiFi.status() != WL_CONNECTED)
    {
        Serial.println("[WiFi] ⚠ Mất kết nối!");
```

```
        delay(5000);
        return;
    }

    sendCount++;

    // Tạo JSON giả giống đúng format backend expect
    JsonDocument doc;
    doc["gateway_id"] = GATEWAY_ID;
    doc["secret"] = GATEWAY_SECRET;

    JsonArray dataArray = doc["data"].to<JsonArray>();
    JsonObject item = dataArray.add<JsonObject>();

    item["node_id"] = "NODE_001";
    item["pm25"] = 12.3 + (sendCount % 10) * 0.5;
    item["pm10"] = 45.6;
    item["co2"] = 800 + sendCount;
    item["tvoc"] = 50;
    item["temperature"] = 27.5;
    item["humidity"] = 65.0;
    item["battery"] = 85;
    item["rssi"] = -67;

    String jsonPayload;
    serializeJson(doc, jsonPayload);

    Serial.printf("[HTTP] #%d Gửi %d bytes...\n", sendCoun
t, jsonPayload.length());

    HTTPClient http;
    http.begin(API_URL);
    http.addHeader("Content-Type", "application/json");
    http.setTimeout(API_TIMEOUT_MS);

    unsigned long txStart = millis();
    int httpCode = http.POST(jsonPayload);
    unsigned long txTime = millis() - txStart;
```

```
    if (httpCode == 200)
    {
        String response = http.getString();
        Serial.printf("[HTTP] ✅ 200 OK (%lu ms)\n", txTim
e);
        Serial.printf("[HTTP]    Response: %s\n", response.
c_str());
    }
    else if (httpCode > 0)
    {
        String response = http.getString();
        Serial.printf("[HTTP] ❌ HTTP %d (%lu ms)\n", httpC
ode, txTime);
        Serial.printf("[HTTP]    Response: %s\n", response.
c_str());
    }
    else
    {
        Serial.printf("[HTTP] ❌ Connection failed: %s (%lu
ms)\n",
                      http.errorToString(httpCode).c_str(),
txTime);
    }

    http.end();
    Serial.println();

    delay(10000);
}
```

💡 Kết quả: post thành công dữ liệu lên server. Server đã xử lý thành công, lưu dữ liệu vào database và gửi phản hồi thành công.
Logs nhận được từ serial:

```
========================================
TEST: HTTP POST → Cloud Server
API: http://192.168.137.1:3000/api/v1/telemetry
Gateway ID: GW_001
```

```
[WiFi] Đang kết nối.
[WiFi] ✅ IP: 192.168.137.208
```

```
[HTTP] Gửi JSON test mỗi 10 giây...
----------------------------------------
[HTTP] #1 Gửi 184 bytes...
[HTTP] ✅ 200 OK (122 ms)
[HTTP]    Response: {"success":true,"message":"Telemetry da
ta ingested successfully"}
```

```
[HTTP] #2 Gửi 184 bytes...
[HTTP] ✅ 200 OK (336 ms)
[HTTP]    Response: {"success":true,"message":"Telemetry da
ta ingested successfully"}
```

```
[HTTP] #3 Gửi 184 bytes...
[HTTP] ✅ 200 OK (105 ms)
[HTTP]    Response: {"success":true,"message":"Telemetry da
ta ingested successfully"}
```

```
[HTTP] #4 Gửi 184 bytes...
[HTTP] ✅ 200 OK (107 ms)
[HTTP]    Response: {"success":true,"message":"Telemetry da
ta ingested successfully"}
```

```
[HTTP] #5 Gửi 184 bytes...
[HTTP] ✅ 200 OK (147 ms)
```

```
[HTTP]    Response: {"success":true,"message":"Telemetry da
ta ingested successfully"}
```

```
[HTTP] #6 Gửi 184 bytes...
[HTTP] ✅ 200 OK (105 ms)
[HTTP]    Response: {"success":true,"message":"Telemetry da
ta ingested successfully"}
```

Test của SensorNode

Test PMS7003

File test_pms7003.cpp:

```
/**
 * TEST: PMS7003 - Cảm biến bụi mịn PM2.5 / PM10
 * Giao tiếp: UART2 (RX=16, TX=17), SET pin (GPIO15)
 *
 * Flash: pio run -e test_pms7003 -t upload && pio device m
onitor
 *
 * PASS khi: Quạt quay khi SET=HIGH, PM2.5/PM10 > 0 sau war
m-up
 *
 * Chu kỳ test: Bật quạt → warm-up 30s → đọc 3 lần → tắt qu
ạt → chờ 30s
 */

#include <Arduino.h>
#include <PMS.h>
#include "config.h"

static HardwareSerial pmsSerial(2);
static PMS pms(pmsSerial);
static PMS::DATA pmsData;

void setup() {
    Serial.begin(115200);
```

```
    delay(1000);

    Serial.println();
    Serial.println("========================================
=");
    Serial.println("  TEST: PMS7003 (PM2.5 / PM10)");
    Serial.printf("  UART: RX=%d, TX=%d\n", PMS_RX_PIN, PMS
_TX_PIN);
    Serial.printf("  SET Pin: GPIO%d\n", PMS_SET_PIN);
    Serial.println("========================================
=");
    Serial.println();

    // Init UART2
    pmsSerial.begin(9600, SERIAL_8N1, PMS_RX_PIN, PMS_TX_PI
N);
    Serial.println("[PMS7003] UART2 khởi tạo OK (9600 bau
d)");

    // Init SET pin
    pinMode(PMS_SET_PIN, OUTPUT);
    digitalWrite(PMS_SET_PIN, HIGH);
    Serial.printf("[PMS7003] SET pin (GPIO%d) = HIGH → Quạt
chạy\n", PMS_SET_PIN);

    // Passive mode
    pms.passiveMode();
    Serial.println("[PMS7003] Chế´độ: Passive (đọc khi yêu
câù)");

    Serial.println();
    Serial.println("[PMS7003] ✅ INIT OK!");
    Serial.println("[PMS7003] Bắt đầù chu kỳ test...");
    Serial.println();
}

static uint16_t cycleCount = 0;
```

```
void loop() {
    cycleCount++;

    // ═══ Bước 1: Bật quạt ═══
    Serial.println("═══════════════════════════════════════
=");
    Serial.printf("  CHU KỲ #%d\n", cycleCount);
    Serial.println("═══════════════════════════════════════
=");

    Serial.println("[PMS7003] Bật quạt (SET=HIGH, wakeU
p)");
    digitalWrite(PMS_SET_PIN, HIGH);
    pms.wakeUp();

    // ═══ Bước 2: Warm-up 30 giây (đếm ngược) ═══
    Serial.printf("[PMS7003] Warm-up %d giây", PMS_WARMUP_M
S / 1000);
    for (int i = PMS_WARMUP_MS / 1000; i > 0; i -= 5) {
        Serial.printf(".");
        delay(5000);
    }
    Serial.println(" OK!");

    // ═══ Bước 3: Đọc 3 lần liên tiếp ═══
    Serial.println();
    Serial.println("  #  │  PM1.0  │  PM2.5  │  PM10   │ Kế
t quả");
    Serial.println("──────────────────────────────────────
──────");

    for (int i = 1; i <= 3; i++) {
        pms.requestRead();

        if (pms.readUntil(pmsData, 3000)) {
            uint16_t pm10_std  = pmsData.PM_AE_UG_1_0;
            uint16_t pm25_std  = pmsData.PM_AE_UG_2_5;
            uint16_t pm100_std = pmsData.PM_AE_UG_10_0;
```

```
            bool dataOk = (pm25_std > 0 || pm100_std > 0);
            const char* status = dataOk ? "✅ PASS" : "⚠ ZE
RO";

            Serial.printf("  %d  | %3d µg  | %3d µg  | %3d
µg  | %s\n",
                          i, pm10_std, pm25_std, pm100_st
d, status);
        } else {
            Serial.printf("  %d  |  ERROR  |  ERROR  |  ERR
OR  | ❌ FAIL\n", i);
        }

        delay(2000);
    }

    // ═══ Bước 4: Test SET pin (tắt quạt) ═══
    Serial.println();
    Serial.println("[PMS7003] Tắt quạt (sleep, SET=LOW)");
    pms.sleep();
    digitalWrite(PMS_SET_PIN, LOW);

    Serial.println("[PMS7003] → Kiểm tra: quạt phải NGỪNG q
uay");
    Serial.println("[PMS7003]   (Nếu quạt vẫn quay → SET pi
n chưa nối hoặc lỏi)");

    // Chờ 30 giây trước chu kỳ tiếp theo
    Serial.println();
    Serial.println("[PMS7003] Chờ 30 giây trước chu kỳ tiếp
p...");
    delay(30000);
}
```

💡 Kết quả: Sử dụng PMS7003 để đo thành công các chỉ số bụi mịn pm1.0, pm2.5, pm10.

```
Logs nhận được khi chạy file test:
========================================
TEST: PMS7003 (PM2.5 / PM10)
UART: RX=16, TX=17
SET Pin: GPIO15
```

```
[PMS7003] UART2 khởi tạo OK (9600 baud)
[PMS7003] SET pin (GPIO15) = HIGH → Quạt chạy
[PMS7003] Chế´độ: Passive (đọc khi yêu câù)
```

```
[PMS7003] ✅ INIT OK!
[PMS7003] Bắt đâù chu kỳ test...
```

```
════════════════════════════════════════
CHU KỲ #1
════════════════════════════════════════
[PMS7003] Bật quạt (SET=HIGH, wakeUp)
[PMS7003] Warm-up 30 giây..... OK!
```

```
| PM1.0 | PM2.5 | PM10 | Kết quả
```

```
──────────────────────────────────────────────────
1  |  33 µg  |  54 µg  |  67 µg  | ✅ PASS
2  |  33 µg  |  54 µg  |  67 µg  | ✅ PASS
3  |  33 µg  |  54 µg  |  67 µg  | ✅ PASS
µg  | ✅ PASS
```

```
[PMS7003] Tắt quạt (sleep, SET=LOW)
[PMS7003] → Kiểm tra: quạt phải NGỪNG quay
[PMS7003]   (Nếú quạt vâñ quay → SET pin chưa nôí hoặc lôĩ)
```

```
[PMS7003] Chờ 30 giây trước chu kỳ tiếp...
════════════════════════════════════════
```

```
CHU KỲ #2
═══════════════════════════════════════════
[PMS7003] Bật quạt (SET=HIGH, wakeUp)
═══════════════════════════════════════════
═══════════════════════════════════════════
═══════════════════════════════════════════
[PMS7003] Bật quạt (SET=HIGH, wakeUp)
[PMS7003] Warm-up 30 giây...... OK!
[PMS7003] Warm-up 30 giây...... OK!
```

```
| PM1.0 | PM2.5 | PM10 | Kết quả
```

```
| PM1.0 | PM2.5 | PM10 | Kết quả
```

```
| PM1.0 | PM2.5 | PM10 | Kết quả
```

```
──────────────────────────────────────────────
──────────────────────────────────────────────
1  |  32 µg  |  52 µg  |  65 µg  | ✅ PASS
1  |  32 µg  |  52 µg  |  65 µg  | ✅ PASS
2  |  32 µg  |  52 µg  |  65 µg  | ✅ PASS
3  |  32 µg  |  52 µg  |  65 µg  | ✅ PASS
```

```
[PMS7003] Tắt quạt (sleep, SET=LOW)
[PMS7003] → Kiểm tra: quạt phải NGỪNG quay
[PMS7003]   (Nếu quạt vẫn quay → SET pin chưa nối hoặc lỗi)
```

```
[PMS7003] Chờ 30 giây trước chu kỳ tiếp...
```